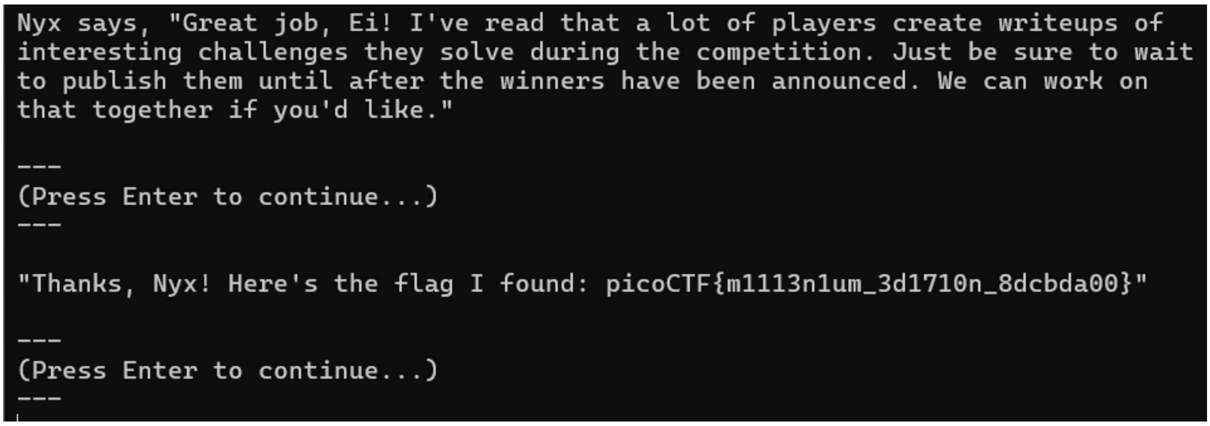
PicoCTF 2025

1 FANTASY CTF

Play through the story

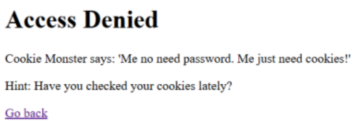
```
Nyx brings up the registration page.

Options:
A) *Register multiple accounts*
B) *Share an account with a friend*
C) *Register a single, private account*
[a/b/c] > c

Nyx nods, "Good choice, Ei. Let's proceed with a single, private account. It
can be tempting to try and do something 'clever', but registering multiple
accounts or sharing an account with a friend can lead to disqualification."

---
(Press Enter to continue...)
---

"Ok," Nyx says, "Registering you for the competition... There's an introductory
audio message, piping to your speakers."
```

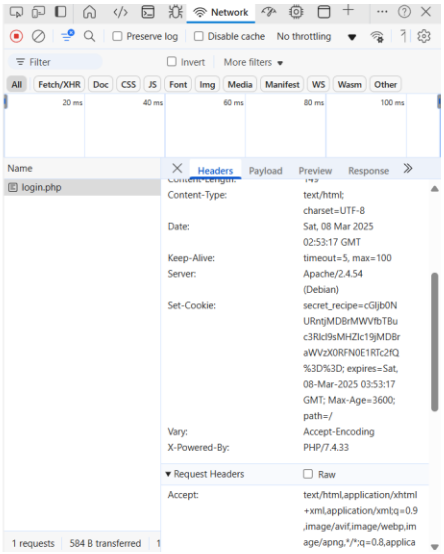
```
"Oh interesting," Eibhilin says, "It seems like the sanity challenge is an old
school interactive fiction game."

---
(Press Enter to continue...)
---

Options:
A) *Play the game*
B) *Search the Ether for the flag*
[a/b] > a

"Good choice, Ei," Nyx says, "You never want to share flags or artifact
downloads."

---
(Press Enter to continue...)
---

 Playing the Game
Playing the Game: 100%|██████████████████████████████| [time left: 00:00]
Playing the Game completed successfully!

---
(Press Enter to continue...)
---

"That was fun!" Eibhilin exclaims, "I found the flag!"
```



Flag: picoCTF{m1113n1um_3d1710n_8dcbda00

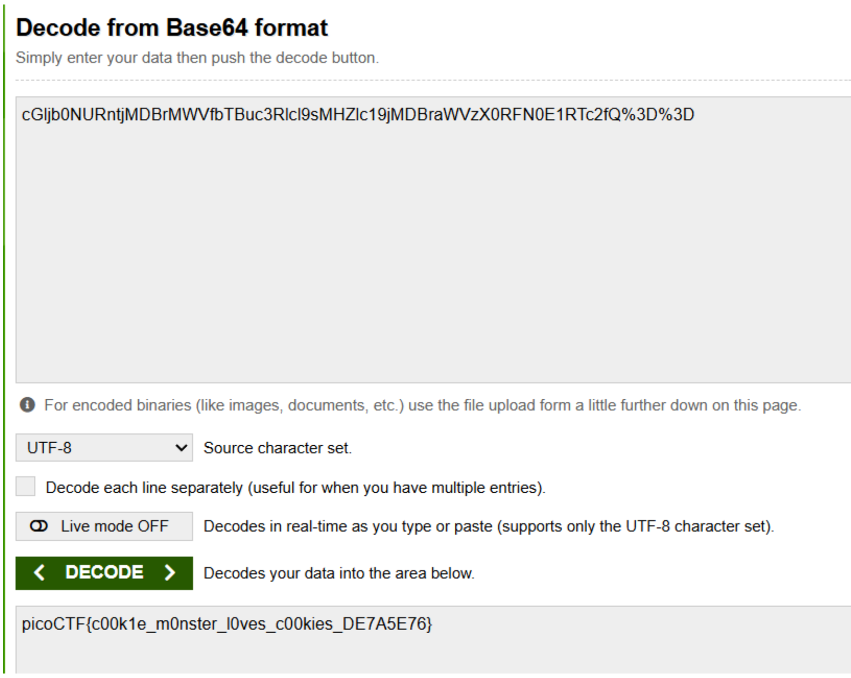
2 COOKIE MONSTER SECRET RECIPE





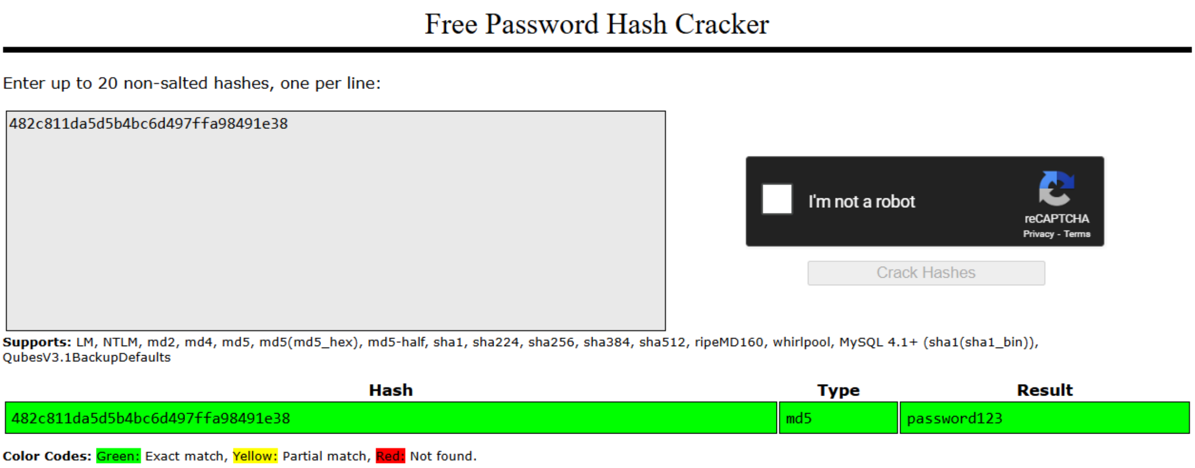
Inspect cookies to find flag.

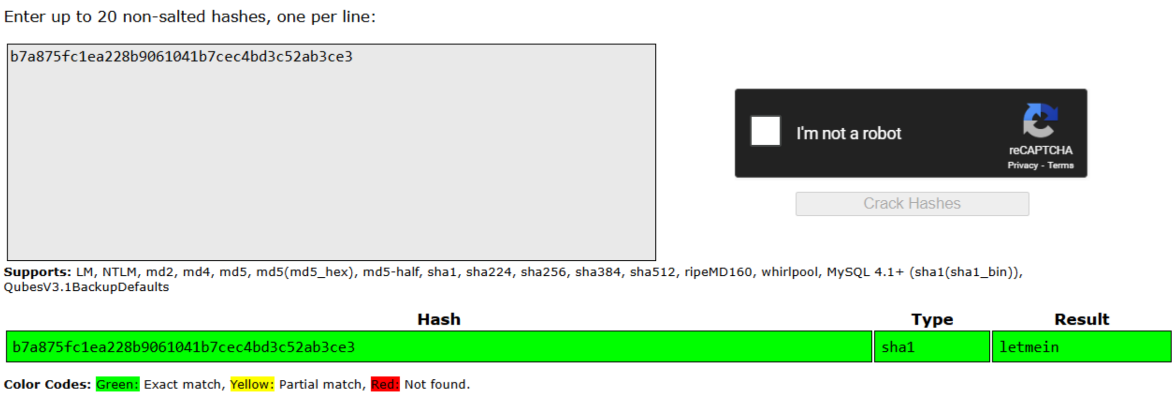
Decode from Base 64 format to text.



Flag: picoCTF{c00k1e_m0nster_l0ves_c00kies_DE7A5E76}

3 HASHCRACK





Enter up to 20 non-salted hashes, one per line:

```
916e8c4f79b25028c9e467f1eb8eee6d6bbdff965f9928310ad30a8d88697745
```

I'm not a robot reCAPTCHA Privacy - Terms

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
916e8c4f79b25028c9e467f1eb8eee6d6bbdff965f9928310ad30a8d88697745	sha256	qwerty098

Color Codes: Green: Exact match, Yellow: Partial match, Red: Not found.

```
ameenah@AmeenahsASUS:~$ nc verbal-sleep.picoctf.net 52014
Welcome!! Looking For the Secret?

We have identified a hash: 482c811da5d5b4bc6d497ffa98491e38
Enter the password for identified hash: password123
Correct! You've cracked the MD5 hash with no secret found!

Flag is yet to be revealed!! Crack this hash: b7a875fc1ea228b9061041b7cec4bd3c52ab3ce3
Enter the password for the identified hash: letmein
Correct! You've cracked the SHA-1 hash with no secret found!

Almost there!! Crack this hash: 916e8c4f79b25028c9e467f1eb8eee6d6bbdff965f9928310ad30a8d88697745
Enter the password for the identified hash: qwerty098
Correct! You've cracked the SHA-256 hash with a secret found.
The flag is: picoCTF{UseStr0nG_h@shEs_&PaSswDs!_7f29c9da}
```

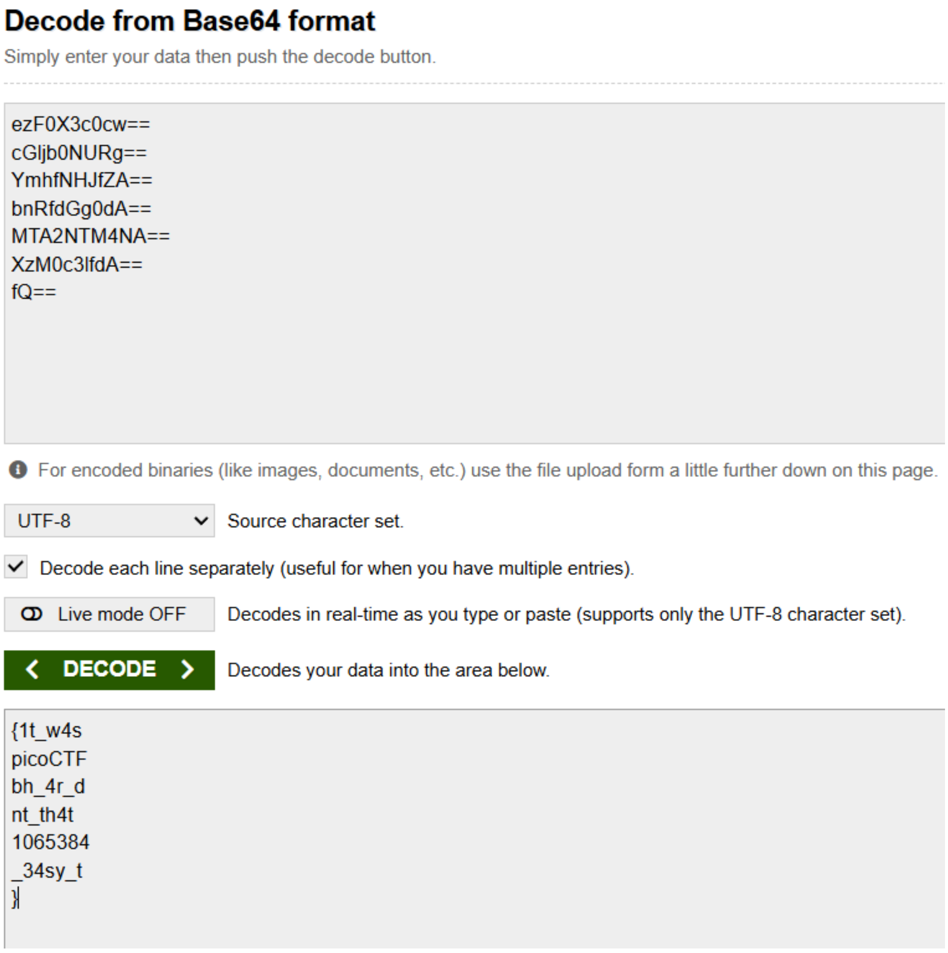
Unhash each hash string to find the password for each stage. The flag is then revealed.

Flag: picoCTF{UseStr0nG_h@shEs_&PaSswDs!_7f29c9da}

4 PHANTOM INTRUDER

```
ameenah@AmeenahsASUS:~$ strings myNetworkTraffic.pcap
ezF0X3c0cw==O
I+znCJg=O
/9QZRtc=O
uF+2UTs=O
pDW7rkI=O
8esVOK4=O
cGljb0NURg==O
KWEh2jQ=O
BGAVCe8=O
RA+7xFw=O
YmhfNHJfZA==O
CMEx344=O
PGb6oYA=O
uSy5rvo=O
bnRfdGg0dA==O
MTA2NTM4NA==O
xIJPbWg=O
XzM0c3lfdA==O
oDis5T8=O
v2XglLs=O
fQ==O
STneebY=
```

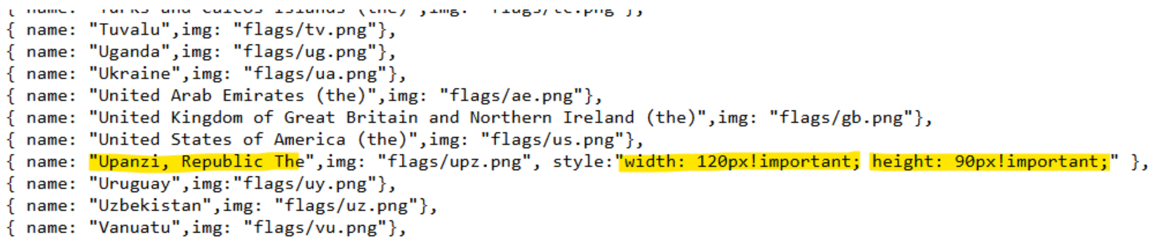
Strings of pcap files are analysed. The strings that end with two equal signs are decoded from base64 format. The resulting strings are rearranged to form the flag.



Flag: picoCTF{1t_w4snt_th4t_34sy_tbh_4r_d1065384}

5 FLAGS ARE STEPIC (INCOMPLETE)

I know the flag is hidden in the upz.png file. It is 1.7MB large while the other images are 600KB. And it's not a country. But I can't extract useful information. Strings, ls and binwalk are not giving me any results.

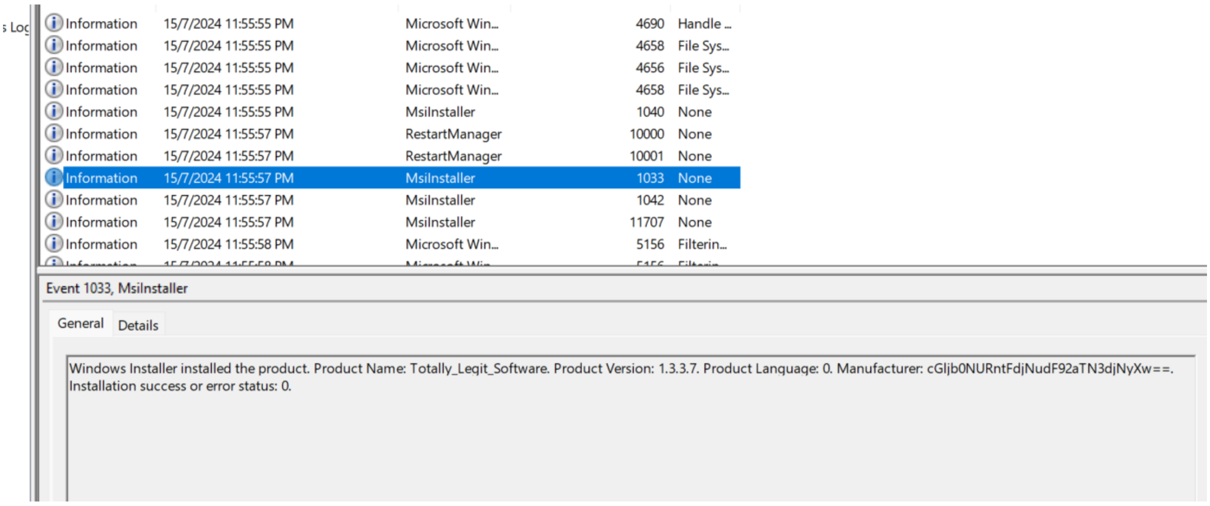


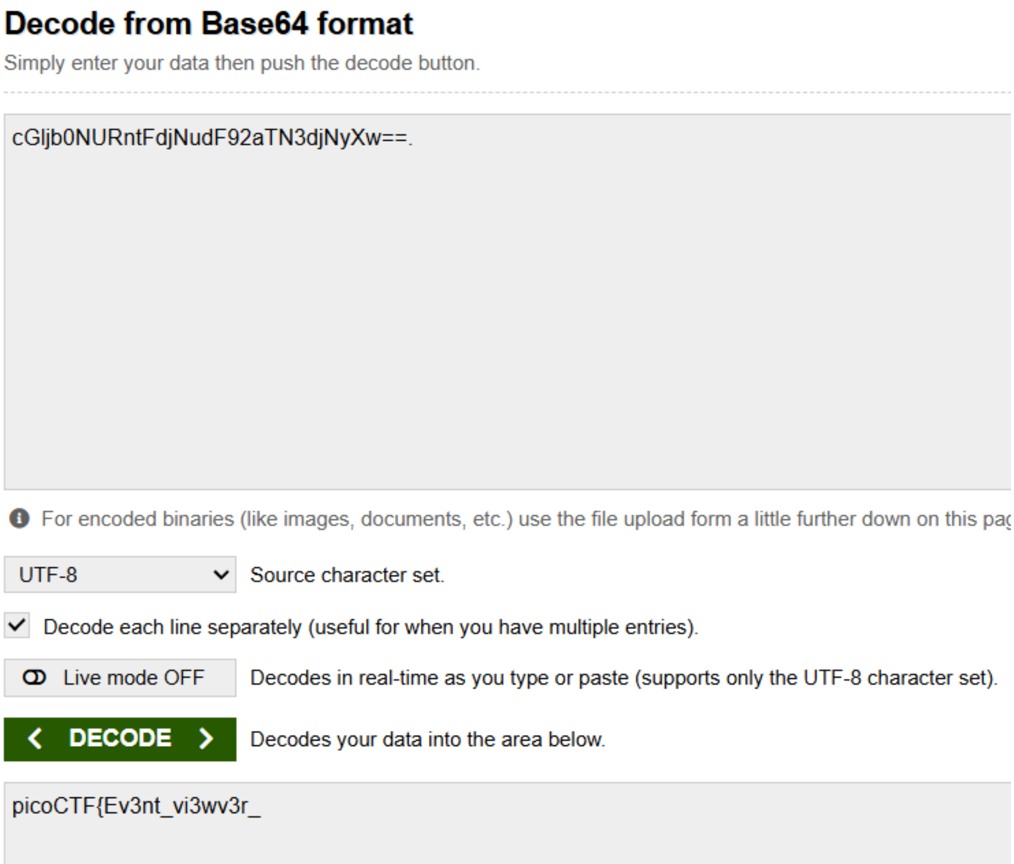
```
{ name: "Tuvalu",img: "flags/tv.png"},
{ name: "Uganda",img: "flags/ug.png"},
{ name: "Ukraine",img: "flags/ua.png"},
{ name: "United Arab Emirates (the)",img: "flags/ae.png"},
{ name: "United Kingdom of Great Britain and Northern Ireland (the)",img: "flags/gb.png"},
{ name: "United States of America (the)",img: "flags/us.png"},
{ name: "Upanzi, Republic The",img: "flags/upz.png", style:"width: 120px!important; height: 90px!important;" },
{ name: "Uruguay",img:"flags/uy.png"},
{ name: "Uzbekistan",img: "flags/uz.png"},
{ name: "Vanuatu",img: "flags/vu.png"},
```

6 EVENT VIEWING

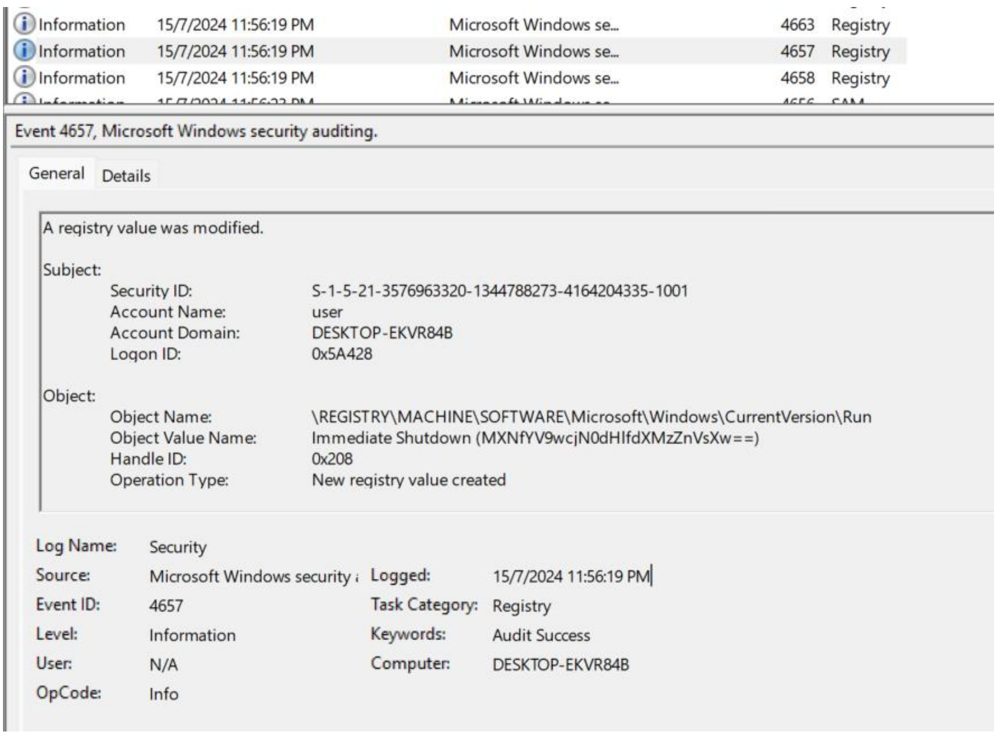
Open up log file.

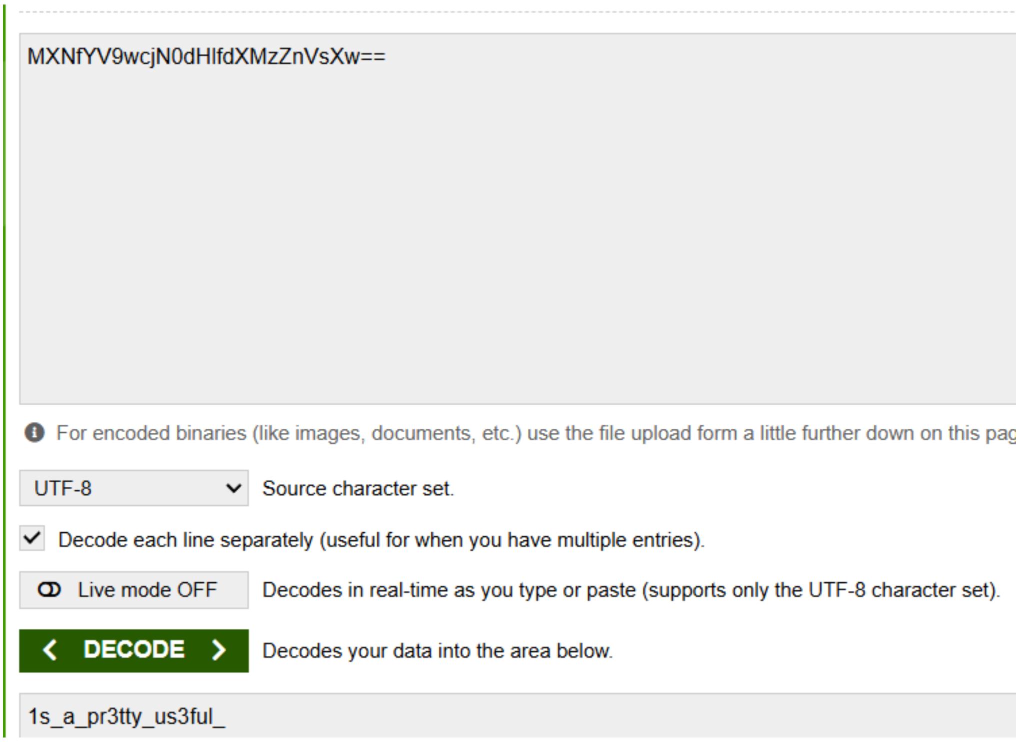
First log has installation information for Totally_Legit_Software is at 11:55:57PM. First third of flag was in base64 format.



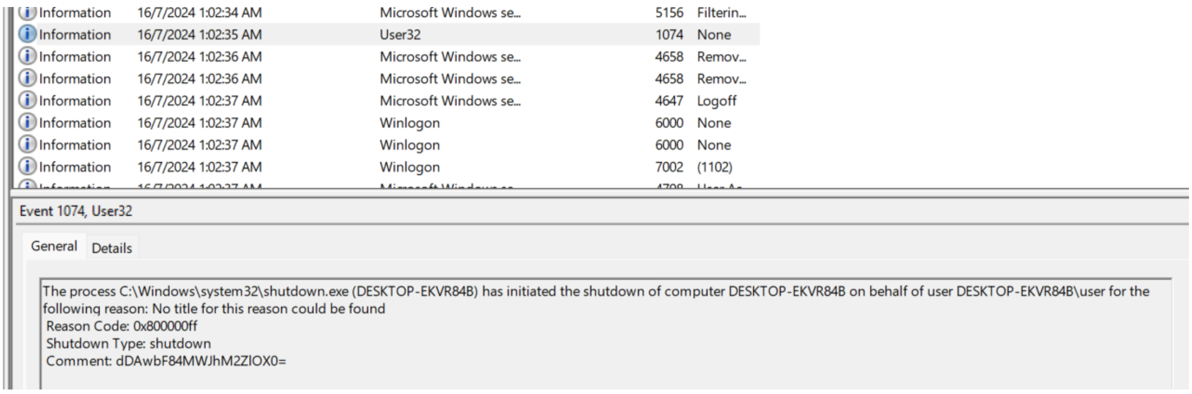


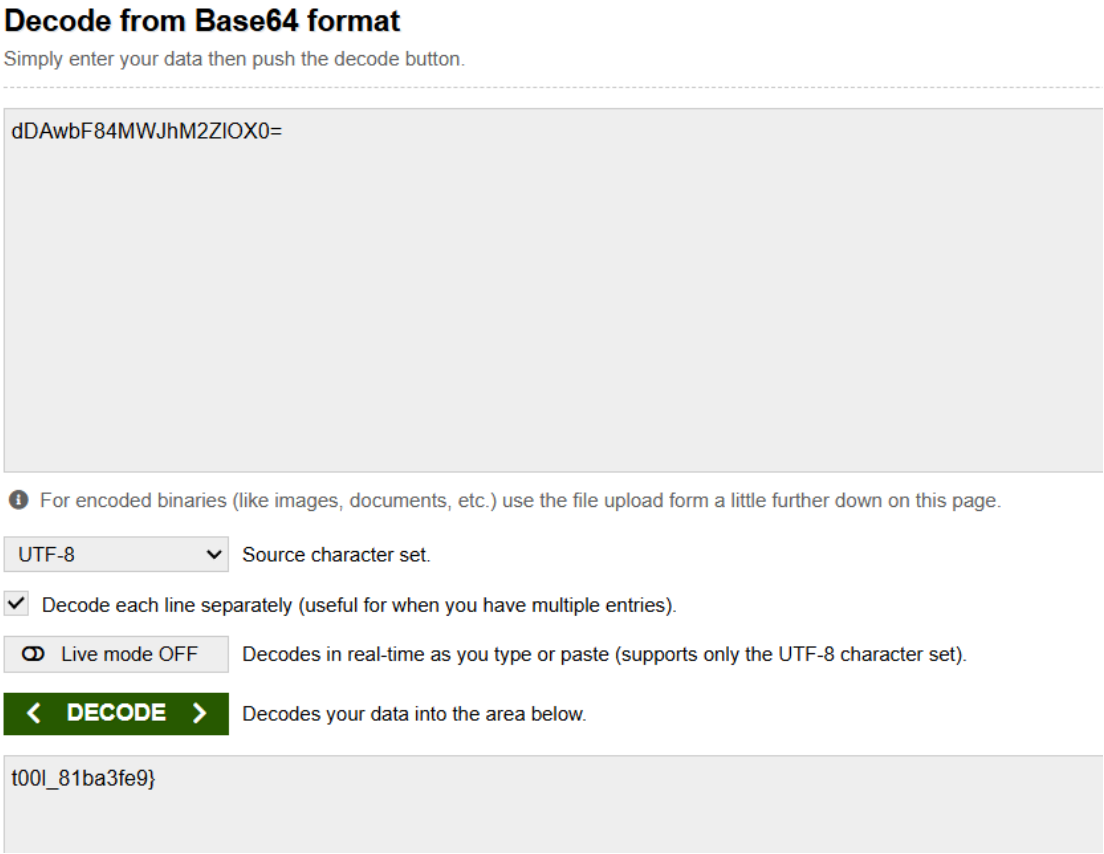
Scrolling through the following logs. I found another part of the flag in base64 format.





No luck running through times right after first shutdown. Scrolled to an hour later.





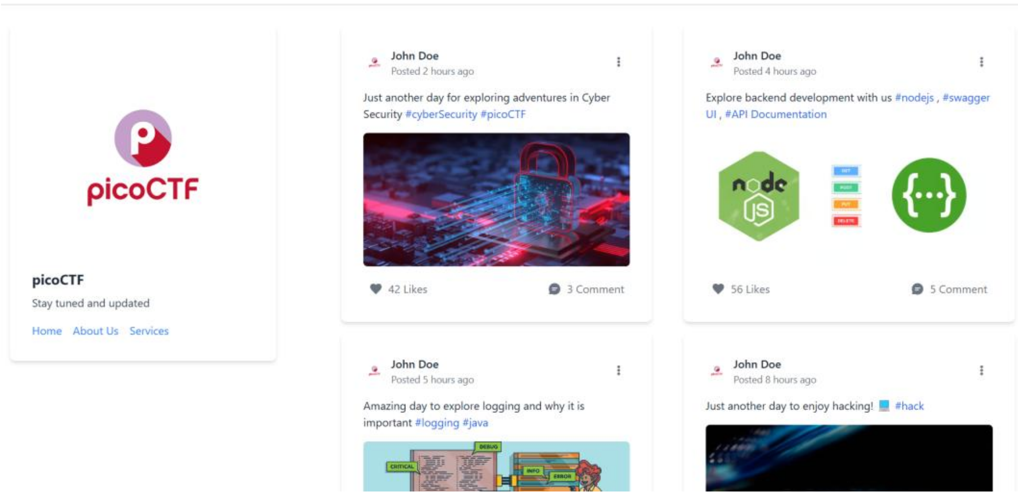
Flag: picoCTF{Ev3nt_vi3wv3r_1s_a_pr3tty_us3ful_t00l_81ba3fe9}

7 EVEN RSA CAN BE BROKEN???

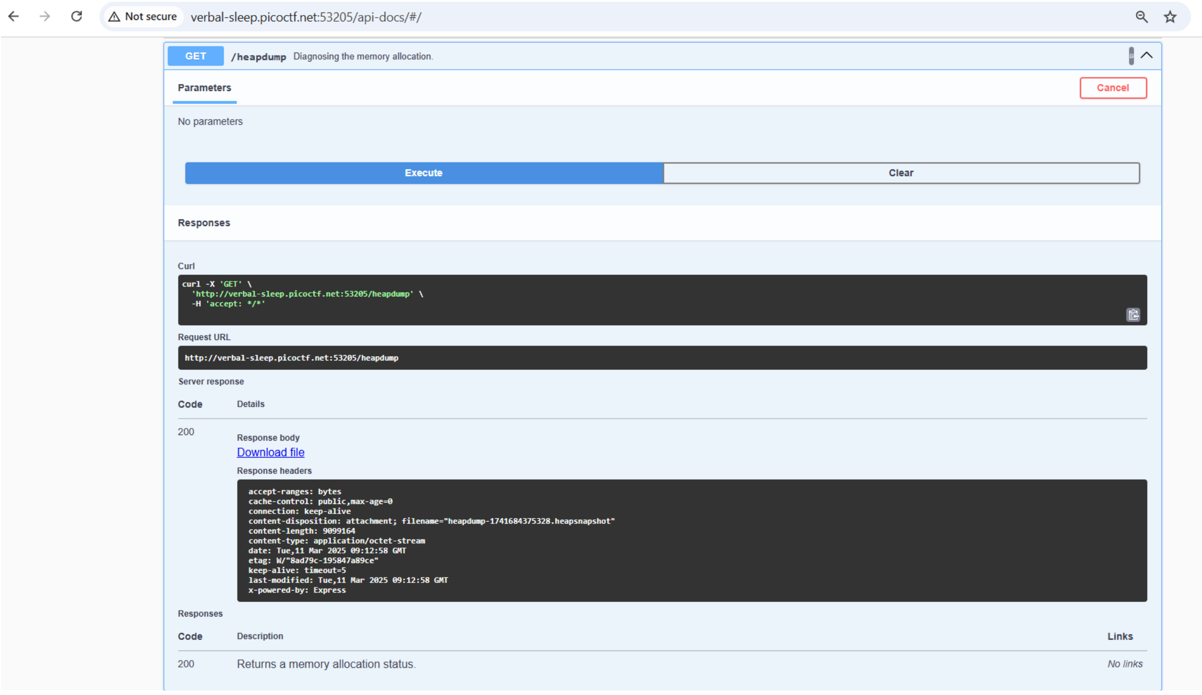
Insert e, N and ciphertext into website and decrypt to reveal flag.

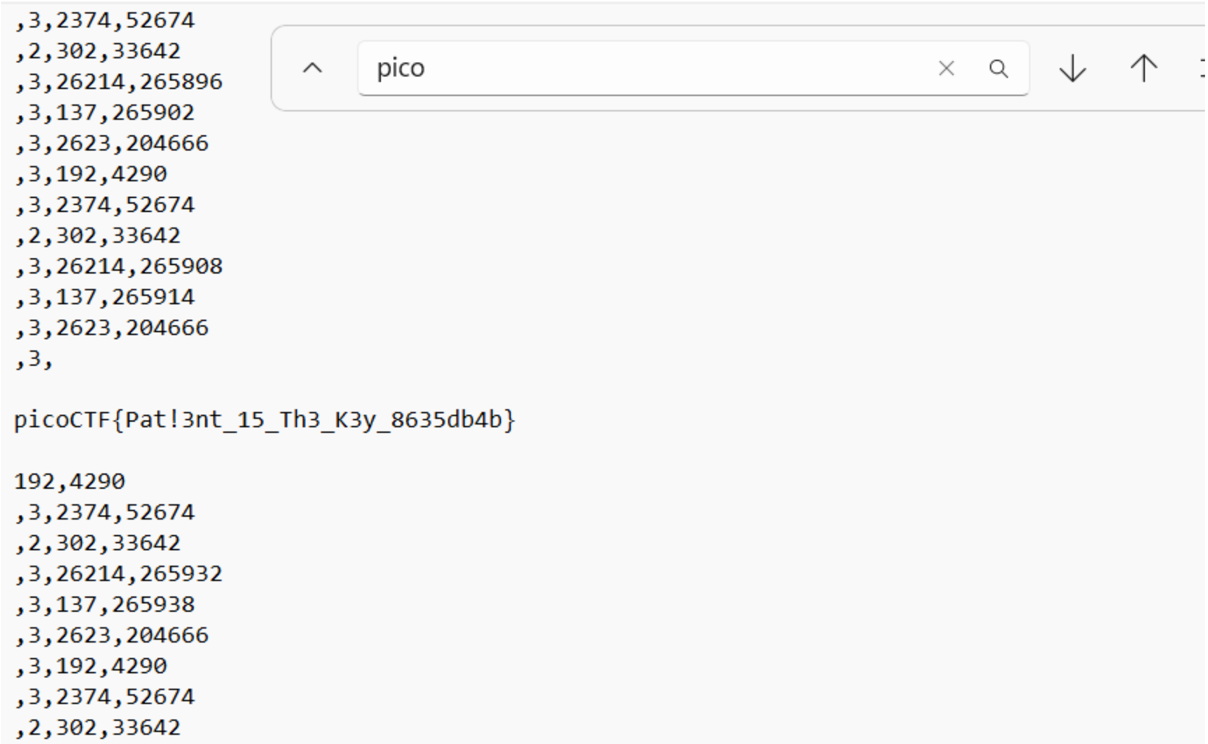
Flag: picoCTF{tw0_1$_pr!m3f81fef0a}

8 HEAD-DUMP



Click on the API documentation link. Once on the page, execute heapdump and download the file. Open it in notepad and search "pico". This reveals the flag.





Flag: picoCTF{Pat!3nt_15_Th3_K3y_8635db4b}

9 QUANTUM SCRAMBLER

After studying the pattern of quantum scrambling for a different numbers of letters, I noticed repetition begins after 4 letters. Repetition increases in powers of two for each increase in letter.

Mapped the word “pico” to its hexadecimal characters to start off. Then replaced remaining hexadecimal values. Sample shown below.

[['p', 'i'], ['c', [], 'o'],

[C, [['p', 'i']], 'T'],

['F', [['p', 'i'], ['c', [], 'o']], '{'],

['p', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T']], 'y'],

['t', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{']], 'h']

, ['o', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{'], ['p', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T']], 'y']], 'n'],

['_', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{'], ['p', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T']], 'y'], ['t', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{']], 'h']], 'i'],

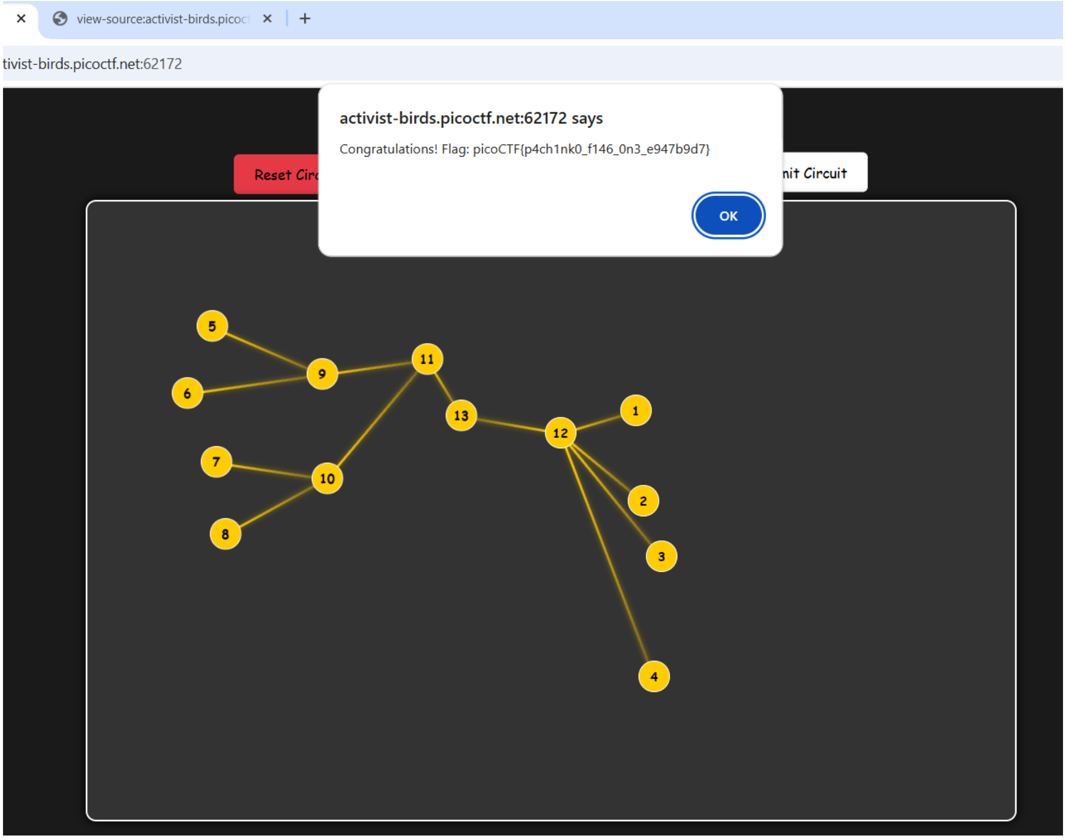
[s, [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{'], ['p', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T']], 'y'], ['t', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{']], 'h'], ['o', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{'], ['p', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T']], 'y']], 'n']], '_'],

[w, [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{'], ['p', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T']], 'y'], ['t', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{']], 'h'], ['o', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{'], ['p', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T']], 'y']], 'n'], ['_', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{'], ['p', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T']], 'y'], ['t', [['p', 'i'], ['c', [], 'o'], ['C', [['p', 'i']], 'T'], ['F', [['p', 'i'], ['c', [], 'o']], '{']], 'h']], 'i']], e],

Flag: picoCTF{python_is_weirdd91a5340}

10 PACHINKO

To understand the game, I looked at the code and did some research about Pachinko. In order to win the game, the inputs node values need to be flipped at the output. This needs to be achieved using the NAND gates which can be added by pressing the “Add intermediate Node” button. I needed to try several circuits until I got the flag.



Flag: picoCTF{p4ch1nk0_f146_0n3_e947b9d7}

11 TAP INTO HASH

The steps used to encrypt the block chain are identified from the block_chain.py file. A random hex chain is first generated and converted to bytes. This is used as the key for the final encryption step. The block chain is then generated and a token is inserted into the middle. In this case the token is the picoCTF flag. Then it is padded to multiple blocks, each sized 16-bytes. Finally, each 16-byte block is XORed with the SHQ-256 hash of the key.

In order to decrypt the given block chain, the encryption steps need to be reversed. A python program is used to do that.

```
C: > Users > ameen > Desktop > Ameenah > Self study > picoCTF 2025 > decrypt_blockchain.py > ...
import hashlib

# Given key and encrypted blockchain
key = b'\x8b\x9a\x00G\xfe\xb3\xf3\x93\xdb\xa8yT\xfe\x15\x87a\xf4\xdf\x00\x8d\xee\xab\xd9\t^|\x04(%\x81\x9e\xf8'
encrypted_blockchain = b'Z5Wo\xe9\xbd\xf4\xed<\xeb=\xcb%\xc4\xf0>S2\x0bl\xe9\xe9\xf0\xe8>\xe8n\x91q\xca\xad=\x01fRo\xbe\xba

# Step 1: XOR Decryption
def xor_bytes(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

def decrypt(ciphertext, key):
    key_hash = hashlib.sha256(key).digest()
    plaintext = b''

    for i in range(0, len(ciphertext), 16):
        block = ciphertext[i:i + 16]
        plaintext += xor_bytes(block, key_hash)

    return plaintext

# Step 2: Remove Padding
def unpad(data):
    padding_length = data[-1]
    return data[:-padding_length]

# Step 3: Decrypt and Unpad
padded_plaintext = decrypt(encrypted_blockchain, key)
plaintext = unpad(padded_plaintext)

# Step 4: Extract the Original Blockchain String
# The flag is inserted in the middle of the plaintext
print("Decrypted Blockchain String:", plaintext.decode('utf-8'))
```

The flag is found in the middle of the decrypted block chain.

```
    padding_length = data[-1]
    return data[:-padding_length]

# Step 3: Decrypt and Unpad
padded_plaintext = decrypt(encrypted_blockchain, key)
plaintext = unpad(padded_plaintext)

# Step 4: Extract the Original Blockchain String
# The flag is inserted in the middle of the plaintext
print("Decrypted Blockchain String:", plaintext.decode('utf-8'))

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  JUPYTER

PS C:\Users\ameen\Desktop\Ameenah\Self study\Python\Drug_discovery> & C:/Python312/python.exe "c:/Users/ameen/Desktop/Ameenah/Self study/picoCTF 2025/decrypt_blockchain.py"
Decrypted Blockchain String: 85dbbeaacffc2894128ab1edae59f6d7cfab5b7995c8c8eef12c8e483cbabb80-008f3cf132d9c72b87ad6ae82c3cccd8685dc34ad5e3992beb3d5af90667773e-001a7039a41bb27460c41f0b044fedbbpicoCTF{block_3SRhViRbT1qcX_XUjM0r49cH_qCzmJZzBK_60647fbb}ce014e7d81f1d569d360c9706a1c98ee-0066a2261eabb27ed179a377bfe4a285d2d8acff133bee199e55a116a5f5a533-00fda7226a6f35003bb8f4c0c6ab004221a892aab38608fbed9f5adc2690b253
PS C:\Users\ameen\Desktop\Ameenah\Self study\Python\Drug_discovery>
```

Flag: picoCTF{block_3SRhViRbT1qcX_XUjM0r49cH_qCzmJZzBK_60647fbb}